Practical No 03

a. Develop an Android application implementing started and bound service.

Project Name: ServiceDemo

FileName: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
    <Button
        android:id="@+id/btnStart"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Service"/>
    <Button
        android:id="@+id/btnStop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Service"
        android:layout_marginTop="15dp"/>
    <Button
        android:id="@+id/btnBound"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bind Service"
        android:layout_marginTop="15dp"/>
</LinearLayout>
```

FileName: MainActivity.java

```
package com.example.servicedemo;


import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.widget.Button;
import android.widget.Toast;


import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {


    Button btnStart, btnStop, btnBound;
```

```
MyBoundService myBoundService;
boolean isBound = false;


@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);


    btnStart = findViewById(R.id.btnStart);
    btnStop = findViewById(R.id.btnStop);
    btnBound = findViewById(R.id.btnBound);


    // Start Service
    btnStart.setOnClickListener(v -> {


        Intent intent = new Intent(
                this,
                MyStartedService.class
        );


        startService(intent);


    });




    // Stop Service
    btnStop.setOnClickListener(v -> {


        Intent intent = new Intent(
                this,
                MyStartedService.class
        );


        stopService(intent);


    });
    // Bind Service
    btnBound.setOnClickListener(v -> {
        Intent intent = new Intent(
                this,
                MyBoundService.class
        );


        bindService(
                intent,
                serviceConnection,
                BIND_AUTO_CREATE
```

```
            );
        });
    }
    // Service Connection
    ServiceConnection serviceConnection = new ServiceConnection() {


        @Override
        public void onServiceConnected(
                ComponentName name,
                IBinder service) {
            MyBoundService.LocalBinder binder =
                    (MyBoundService.LocalBinder) service;
            myBoundService = binder.getService();
            isBound = true;
            Toast.makeText(
                    MainActivity.this,
                    myBoundService.getMessage(),
                    Toast.LENGTH_SHORT
            ).show();
        }


        @Override
        public void onServiceDisconnected(ComponentName name) {
            isBound = false;
        }
    };
    @Override
    protected void onDestroy() {
        super.onDestroy();
        if (isBound) {
            unbindService(serviceConnection);
            isBound = false;
        }
    }
}
```

FileName: MyStartedService.java

```
package com.example.servicedemo;


import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;


public class MyStartedService extends Service {


    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {


        Toast.makeText(
                this,
                "Started Service Running",
                Toast.LENGTH_SHORT
```

```
        ).show();


        return START_STICKY;
    }


    @Override
    public void onDestroy() {
        super.onDestroy();


        Toast.makeText(
                this,
                "Started Service Stopped",
                Toast.LENGTH_SHORT
        ).show();
    }


    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

FileName: MyBoundService.java

```
package com.example.servicedemo;


import android.app.Service;
import android.content.Intent;
import android.os.Binder;
import android.os.IBinder;


public class MyBoundService extends Service {


    private final IBinder binder = new LocalBinder();


    public class LocalBinder extends Binder {
        public MyBoundService getService() {
            return MyBoundService.this;
        }
    }


    @Override
    public IBinder onBind(Intent intent) {
        return binder;
    }


    public String getMessage() {
        return "Bound Service Connected";
    }
}
```

FileName: AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ServiceDemo">

        <!-- Started Service -->
        <service
            android:name=".MyStartedService"
            android:exported="false" />

        <!-- Bound Service -->
        <service
            android:name=".MyBoundService"
            android:exported="false" />

        <!-- Main Activity -->
        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

b. Develop an android application implementing Notification Manager.

ProjectName: Notification Demo

FileName: Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">
    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Notification"/>
</LinearLayout>
```

FileName: MainActivity.java

```
package com.example.notificationdemo;


import android.Manifest;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;


public class MainActivity extends AppCompatActivity {


    private static final String CHANNEL_ID = "MyChannel";
    private static final int NOTIFICATION_PERMISSION_CODE = 100;


    private Button btnNotify;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);


        // Find the button from activity_main.xml
        btnNotify = findViewById(R.id.btnNotify);


        // Create notification channel
        createNotificationChannel();
```

```
        // Request notification permission for Android 13+
        requestNotificationPermission();


        // Button click
        btnNotify.setOnClickListener(v -> {
            showNotification();
        });
    }


    // Request notification permission
    private void requestNotificationPermission() {


        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {


            if
(checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
                    != PackageManager.PERMISSION_GRANTED) {


                requestPermissions(
                        new
String[]{Manifest.permission.POST_NOTIFICATIONS},
                        NOTIFICATION_PERMISSION_CODE
                );
            }
        }
    }


    // Create notification channel
    private void createNotificationChannel() {


        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {


            NotificationChannel channel = new NotificationChannel(
                    CHANNEL_ID,
                    "My Channel",
                    NotificationManager.IMPORTANCE_DEFAULT
            );


            channel.setDescription("Notification Channel for Demo");


            NotificationManager manager =
                    getSystemService(NotificationManager.class);


            if (manager != null) {
                manager.createNotificationChannel(channel);
            }
        }
    }
```

```
    // Show notification
    private void showNotification() {


        // Check permission for Android 13+
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {


            if
(checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
                    != PackageManager.PERMISSION_GRANTED) {


                requestNotificationPermission();
                return;
            }
        }


        NotificationCompat.Builder builder =
                new NotificationCompat.Builder(this, CHANNEL_ID)

.setSmallIcon(android.R.drawable.ic_dialog_info)
                        .setContentTitle("Notification Example")
                        .setContentText("Hello from Android
Notification")

.setPriority(NotificationCompat.PRIORITY_DEFAULT)
                        .setAutoCancel(true);


        NotificationManager manager =
                (NotificationManager)
getSystemService(NOTIFICATION_SERVICE);


        if (manager != null) {
            manager.notify(1, builder.build());
        }
    }
}
```

FileName: AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission
        android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.NotificationDemo">

        <activity
            android:name=".MainActivity"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

c. Develop an android application implementing static and dynamic broadcast receivers.

ProjectName: BroadcastDemo

FileName: Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Broadcast Receiver Example"
        android:textSize="22sp"/>
</LinearLayout>
```

FileName: MainActivity.java

```
package com.example.broadcastdemo;
import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;


public class MainActivity extends AppCompatActivity {


    MyReceiver myReceiver = new MyReceiver();


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);


        // Dynamic Receiver
        IntentFilter filter = new IntentFilter();
        filter.addAction("android.intent.action.AIRPLANE_MODE");


        ContextCompat.registerReceiver(
                this,
                myReceiver,
                filter,
                ContextCompat.RECEIVER_EXPORTED
        );
    }


    @Override
    protected void onDestroy() {
        super.onDestroy();
        unregisterReceiver(myReceiver);
    }
}
```

FileName: MyReceiver.java

```
package com.example.broadcastdemo;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;


public class MyReceiver extends BroadcastReceiver {


    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(
                context,
                "Broadcast Received",
                Toast.LENGTH_SHORT
        ).show();
    }
}
```

FileName: AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.BroadcastDemo">
        <receiver
            android:name=".MyReceiver"
            android:exported="true">
            <intent-filter>
                <action
android:name="android.intent.action.AIRPLANE_MODE" />
            </intent-filter>
        </receiver>

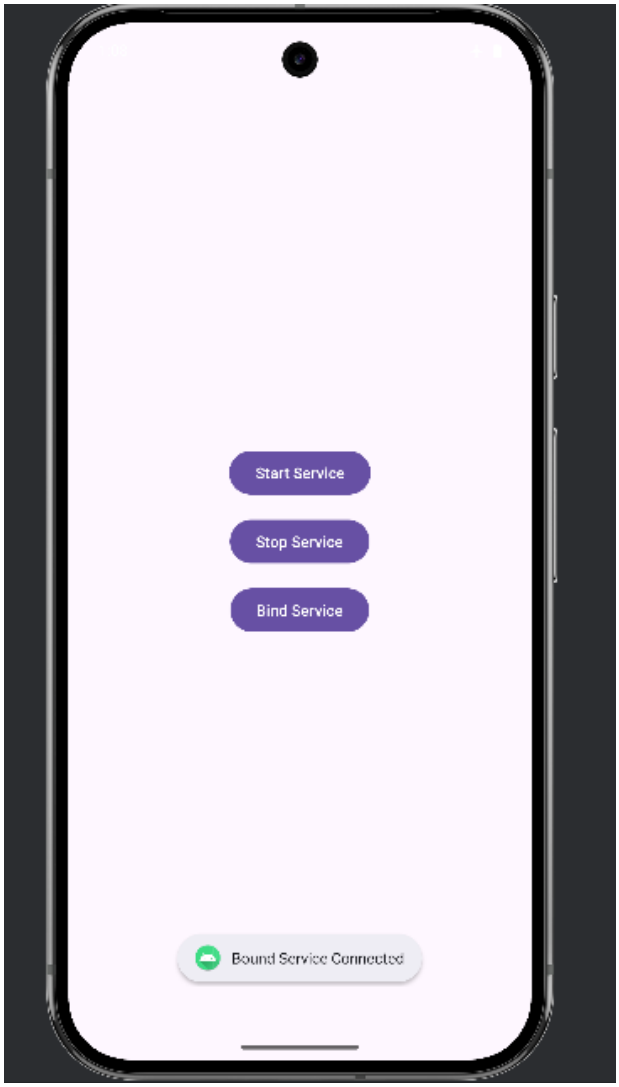
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

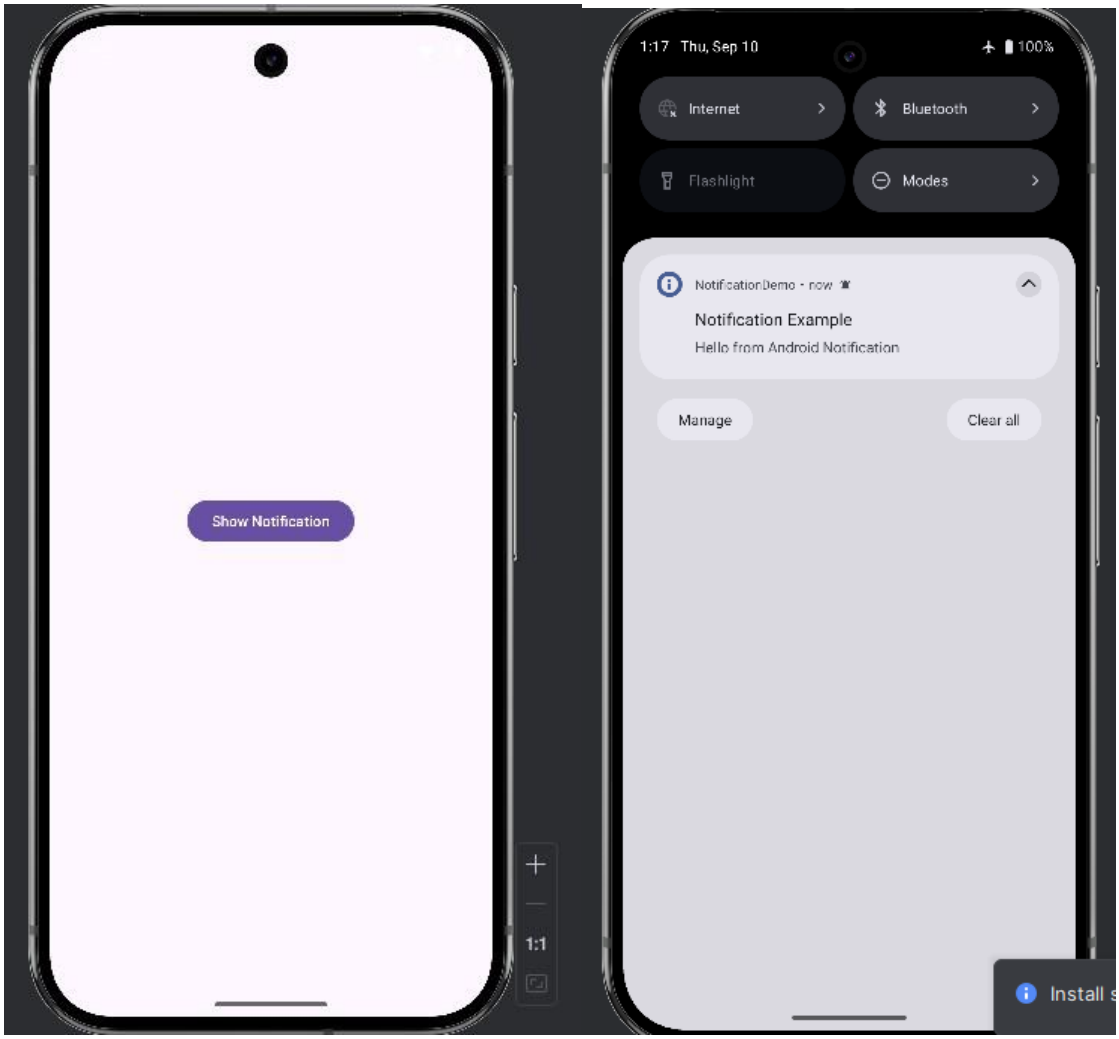
</manifest>
```

Output 3A


Start Service
Stop Service
Bind Service
Started Service Running
Start Service
Stop Service
Bind Service
Started Service Stopped


Start Service
Stop Service
Bind Service
Bound Service Connected

Output 3B


Show Notification
1:17 Thu, Sep 10
100%
Internet
Bluetooth
Flashlight
Modes
NotificationDemo · now
Notification Example
Hello from Android Notification
Manage
Clear all
1:1

Output 3C:

